



郑州大学
Zhengzhou University

第二单元 分组密码

郑州大学网络空间安全学院

王志华

王志华



第2单元 分组密码

提纲

CONTENTS

2.1 分组密码概述

2.2 分组密码分析

2.3 DES

2.4 AES

2.5 SM4

2.6 祖冲之密码

2.7 其他分组密码

2.8 分组密码工作模式

王华



经典分组密码的发展史：

● 美国：

- DES（数据解密标准）：1979-2005，美国政府将其作为美国联邦标准，64比特分组。
- AES（高级加密标准）：2000年至今，美国国家标准与技术研究院（NIST）选择的比利时开发的算法，128比特分组。

● 苏联：

- GOST 28147-89：保密至1990年，至今在用，苏联国家安全委员会（KGB）开发。

● 中国：

- SM4：中国商用分组密码。
- 祖冲之密码（ZUC算法）：被国际组织3GPP推荐为4G无线通信的第三套国际加密和完整性标准的候选算法

2024



分组密码的设计原理：

■ 分组密码的设计原理：

● S盒的设计：

- 设计准则：一般不宜过大。
- 评价指标：非线性度（线性密码分析）、差分均匀性（差分密码分析）、完全性（输出的任一比特与输入的每一比特相关）、**雪崩效应SAC (Strict Avalanche Criterion)**（输入一比特的变动对影响输出一半比特）、扩散特性、可逆性、没有陷门等。

● P盒的设计：

- 设计准则：实现雪崩效应。

● 轮函数的设计：密码结构的核心，依赖S盒

- 设计准则：非线性、**雪崩效应**、**位独立BIC(Bit Independent Criterion)**（输入中一比特的变化和输出中其他比特位的变化是无关的）。
- 评价指标：安全性、~~2022~~速度和灵活性。



分组密码的设计准则：

- 分组密码的设计准则：
 - 迭代的轮数：
 - 设计准则：使密码分析的难度大于简单穷尽搜索攻击的难度。
 - 子密钥的生成方法：迭代分组密码的重要组成部分
 - 设计目标：子密钥的统计独立性和密钥更换的有效性
 - 设计准则：确保密钥和密文符合**雪崩效应**和**位独立**，确保攻击者推导出子密钥和种子密钥的难度很大。
 - 评价指标：软硬件实现简单，速度，不存在简单关系，种子密钥的所有比特对每个子密钥比特的影响大致相同，从一些子密钥比特获得其他子密钥（或者种子密钥）比特在计算上是困难的，没有弱密钥等。

王卫华

2.3.1 DES描述

1、DES概述

- DES (Data Encryption Standard) 是迄今为止世界上最为广泛使用和流行的一种分组密码算法。
- 该算法是在美国NSA (国家安全局) 资助下由IBM公司开发的密码算法, 其初衷是为政府非机密的敏感信息提供较强的加密保护。它是美国政府担保的第一种加密算法并在1977年被正式作为美国联邦信息处理标准。DES主要提供非军事性质的联邦政府机构和私营部门使用, 并迅速成为名声最大, 使用最广的商用密码算法。
- DES的分组长度为64比特, 密钥长度为56比特。

王立华



2、DES发展史

- 1971年，美国IBM公司的 W. Tuchman 和 C. Meyer根据1967年美国Horst Feistel提出的Feistel理论研制成功了Lucifer密码；
- 1972年，美国国家标准局（NBS，现已改为国家标准与技术研究所，NIST）提出了一项保护计算机和数据通信的计划。作为该计划的一部分，要求开发一种单独的标准密码算法；
- 1973年5月15日，美国国家标准局NBS（在联邦记录中**公开征集密码体制**，获胜者将成为美国政府及实际上的产业界标准；
- 1974年8月27日，NBS再次发布征集，同年收到了IBM公司密码研究小组提供的一个算法；

王卫华



NBS发布了公开征集标准密码算法的设计准则

- 算法必须具有较高的安全性
- 算法必须完全确定且易于理解
- 算法安全性必须依赖于密钥，而不是算法本身
- 算法必须对所有的用户都有效
- 算法必须适用于各种应用
- 用以实现算法的电子器件必须很经济
- 算法必须能验证

王卫华



DES的发展史:

- 1975年3月17日, DES首次在联邦记录中公布;
- 1975年8月, NBS征集公众的评论;
- 1976年3月, NBS组织专家对算法进行评估;
- 1976年11月23日, DES被采纳为联邦标准, 用于非密级的政府通信。
- 1977年1月15日, 联邦信息处理标准出版物中 (FIPS PUB46) 给出了DES 的完整描述, 被批准作为美国联邦信息标准;

王卫华



DES的发展史:

- 1977年2月15日，在经过大量的公开讨论后，DES 被采纳作为“非密级”应用的一个标准；
- 1977年7月15日生效；
 - 最初预期，DES 作为一个标准只能使用10-15 年；然而事实证明，DES 要长寿得多
 - 规定每隔5年由美国国家保密局（NSA—National Security Agency）作出评估，并重新批准它是否继续作为联邦加密标准
- 1979年，美国银行协会批准使用DES；
- 1981年，DES成为美国标准化协会（ANSI）标准；
- 1984年2月，ISO成立的数据加密技术委员会（SC20）在DES基础上制定数据加密的国际标准工作；

王华



DES的发展史:

- 1997年, DESCHALL小组找出了DES的密钥, 恢复了密文;
- 1998年5月, 美国EFF宣称用56小时破译了56比特密钥的DES;
- 1998年12月, 美国决定不再使用DES;
 - 因为按照现有的技术水平, 采用不到几十万美元的设备, 就可破开DES密码体制。
- 1999年1月, DES被进行了最后一次评审
 - 在当时, 一个DES的替代品, 高级加密标准 (Advanced Encryption Stan) 已经开始使用了
- 目前的新标准是 AES, 它是由比利时的密码学家Joan Daemen和 Vincent Rijmen设计的分组密码 Rijndael (荣代尔)。

王华



DES的发展趣事:

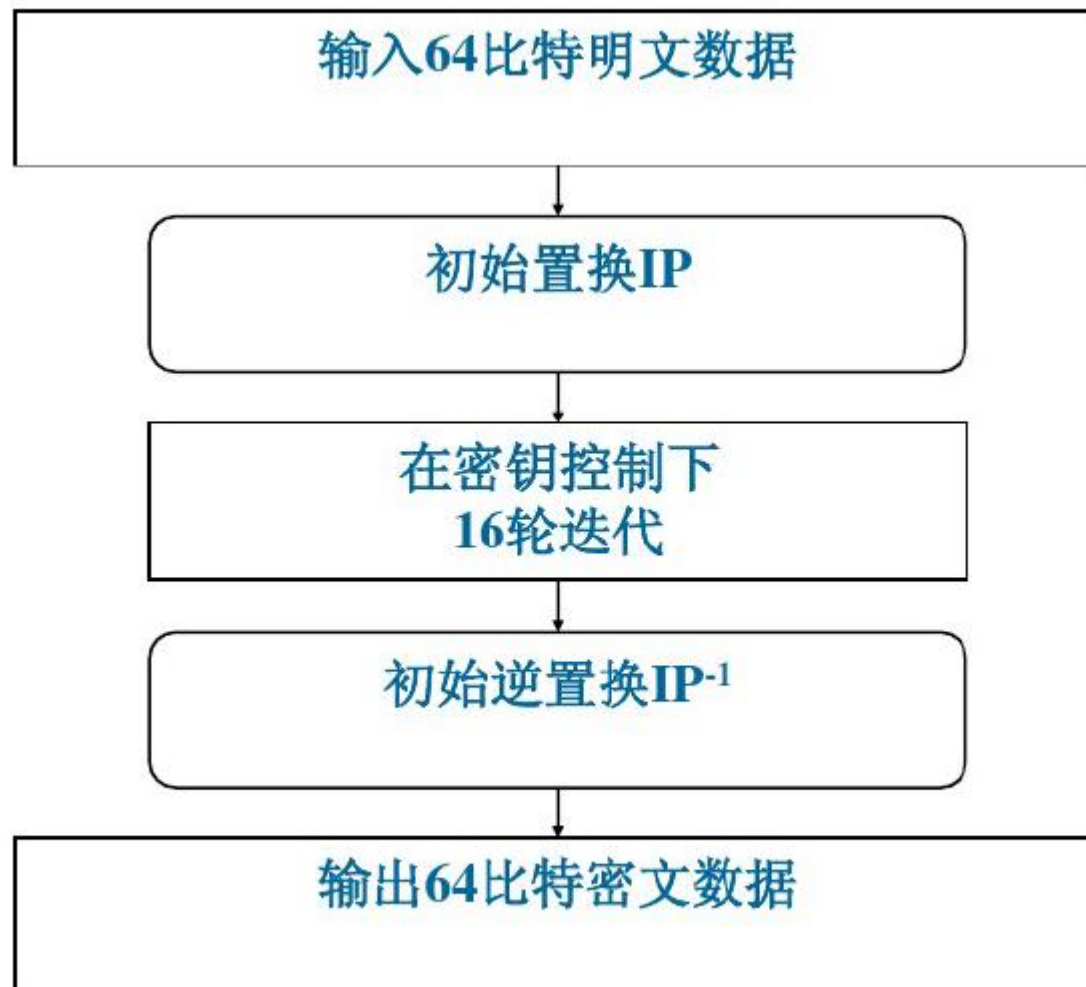
- 由于技术力量不足，NBS即美国国家标准局（现在是美国国家标准技术研究所，即 NIST）求助于NSA (National Security Agency);
- 由于NSA专门负责政府及军方高级敏感信息加密研究，开始不愿参与民间密码开发；
- 后来NSA被怀疑在DES中植入‘后门’，使得NSA很容易破解DES；
- Lucifer算法经过修改成为DES，被指动过手脚
 - 主要修改为将密钥长度由128位减为64位
 - 并且64位中有8位势用来校验奇偶性的，实际密钥长度由为56
 - 破译难度降低了 2^{72} 倍

王立华



3、DES基本思想

- 其中明文分组长为64比特，密钥长为56比特。





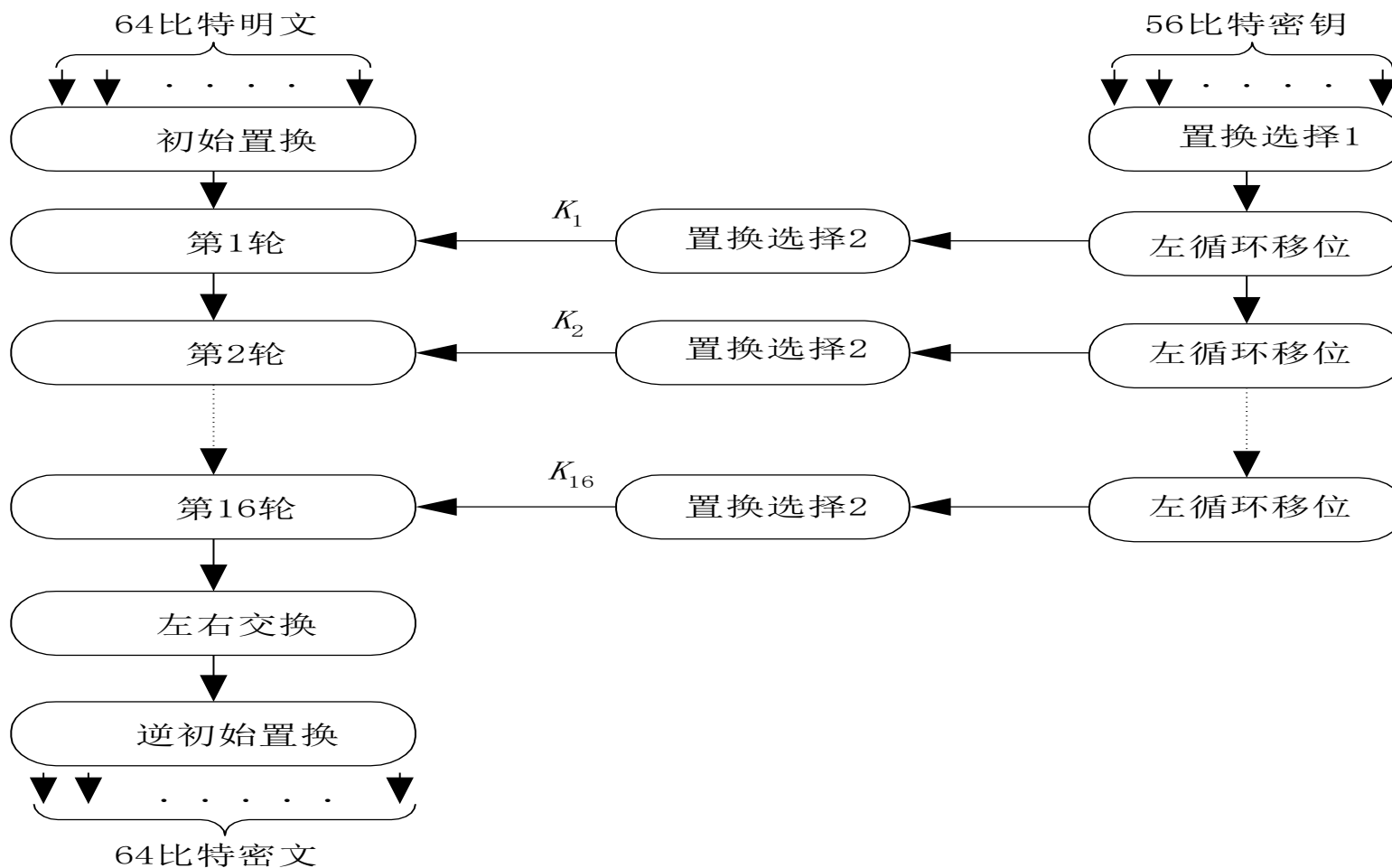
DES算法基本参数:

- 分组加密算法：明文和密文为64比特分组长度
- 对称算法：加密和解密除密钥编排不同外，使用同一算法
- 密钥长度：有效密钥56比特，但每个第8比特为奇偶校验位，可忽略
- 密钥可为任意的56比特数，但存在弱密钥，容易避开
- 采用混淆和扩散的组合，每个组合先代换后置换，共16轮
- 只使用了标准的算术和逻辑运算，易于实现

王卫华



DES算法过程:



2022/3/19



DES加密过程:

- DES结构的左边是明文的处理过程，有三个阶段。
 - 首先是一个**初始置换IP**，用于重排明文分组的64比特数据。
 - 然后是具有相同功能的**16轮变换**，每轮中都有**置换和代换**运算。
 - 第16轮变换的输出分为左右两半，并被**交换次序**。
 - 最后再经过一个**逆初始置换IP⁻¹**（为IP的逆）从而产生64比特的密文。
- ◆ 除初始置换和逆初始置换外，DES的结构和Feistel密码结构完全相同。

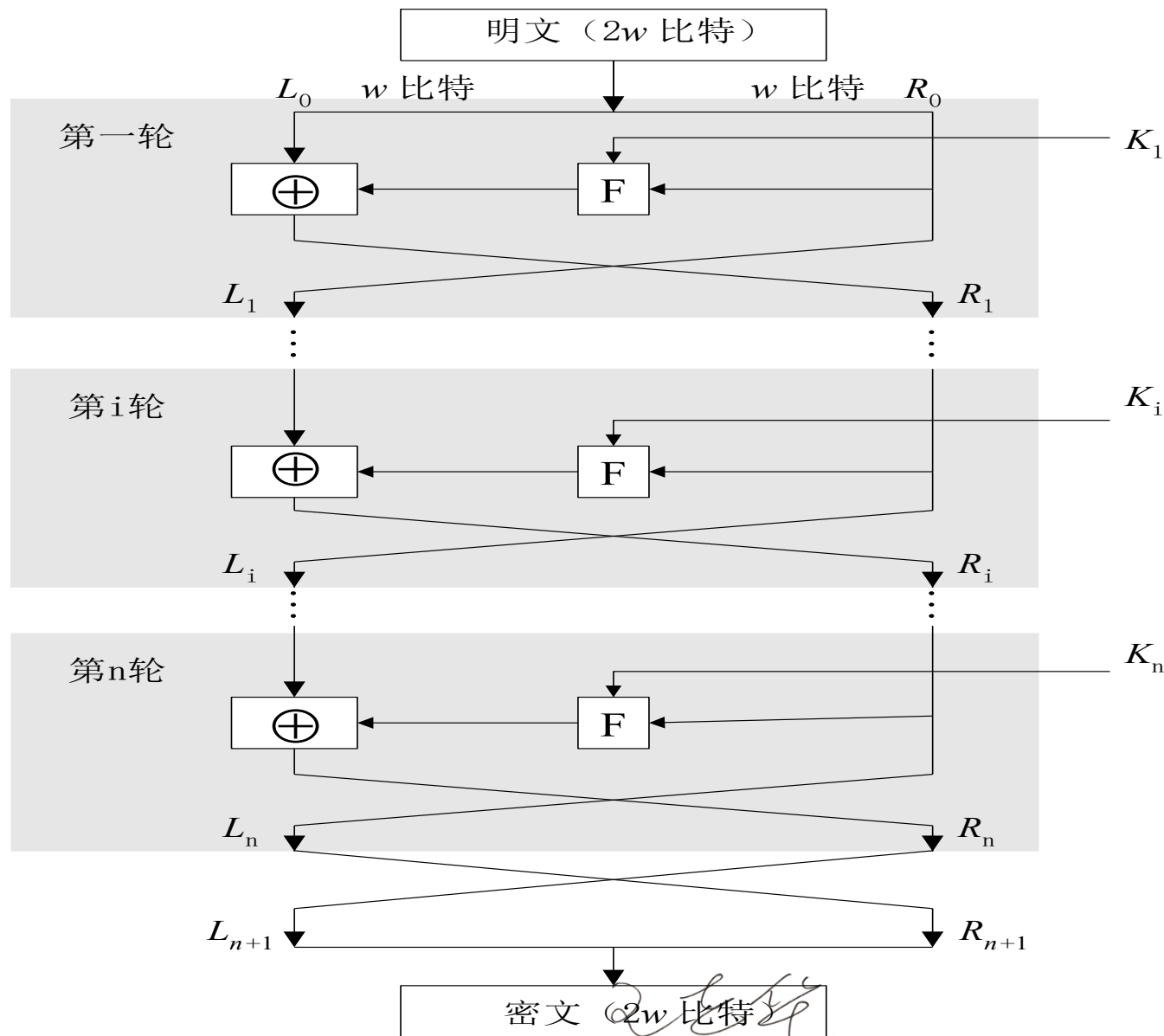
$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

2022



Feistel网络:





DES密钥编排:

- DES结构的右边是使用56比特密钥的方法。
 - 密钥首先通过一个**置换函数**。
 - 然后，对加密过程的每一轮，通过一个**左循环移位**和一个**置换函数**产生一个子密钥。
- ◆ 其中，每轮的置换都相同，但由于密钥被重复迭代，所以产生的每轮子密钥不相同。

王华



4、DES初始置换

- 表3-2 (a) 和表3-2 (b) 分别给出了**初始置换**和**逆初始置换**的定义（从上往下、从左往右编号）。
- 输出的第1比特是输入的第58 比特，该表示方法实现方便。

(a)初始置换IP

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

(b) 逆初始置换IP⁻¹

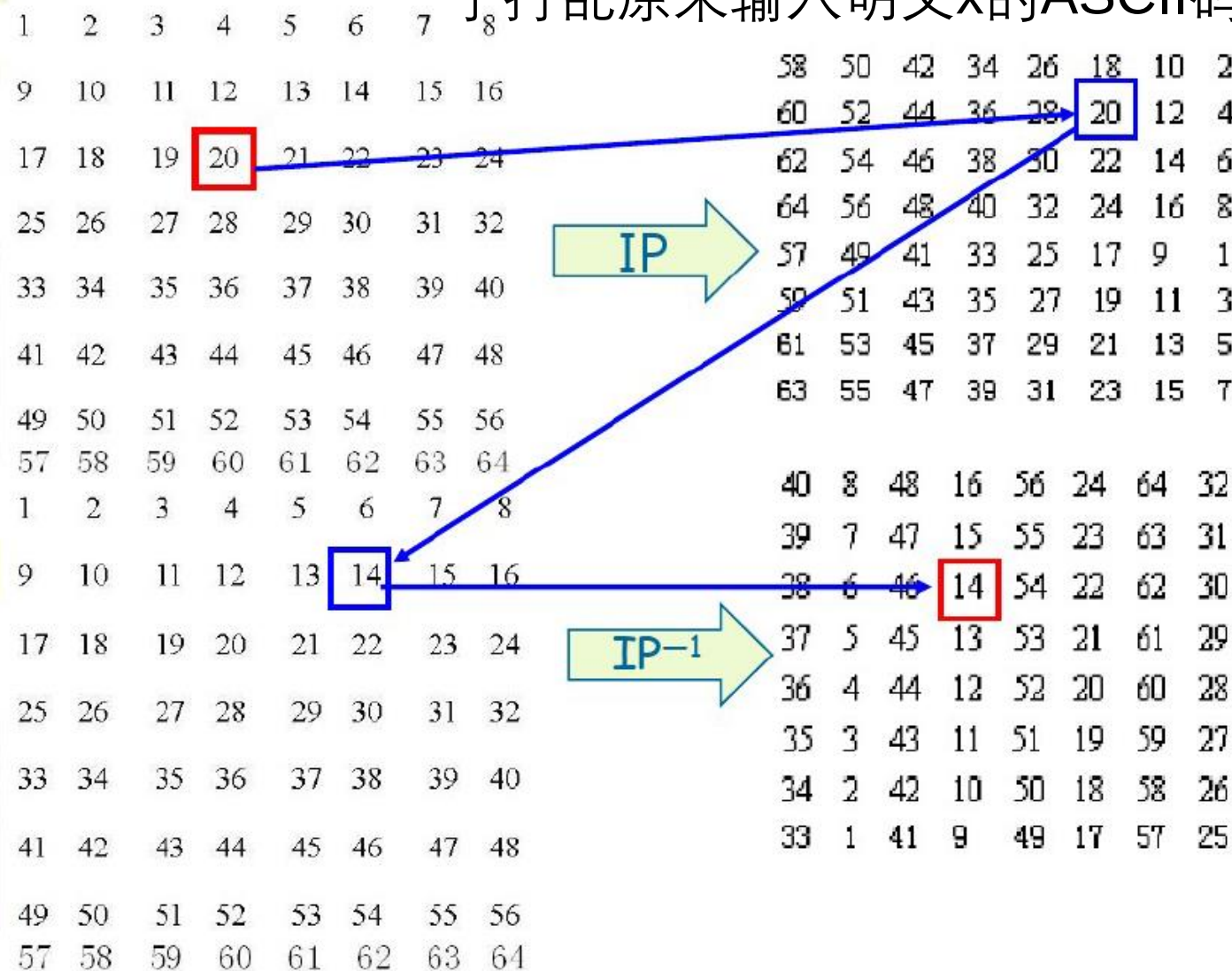
40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

表3-2 DES的置换表



DES初始置换与逆初始置换:

IP 和 IP^{-1} 在密码意义上作用不大, 它们的作用在于打乱原来输入明文 x 的 ASCII 码字划分的关系

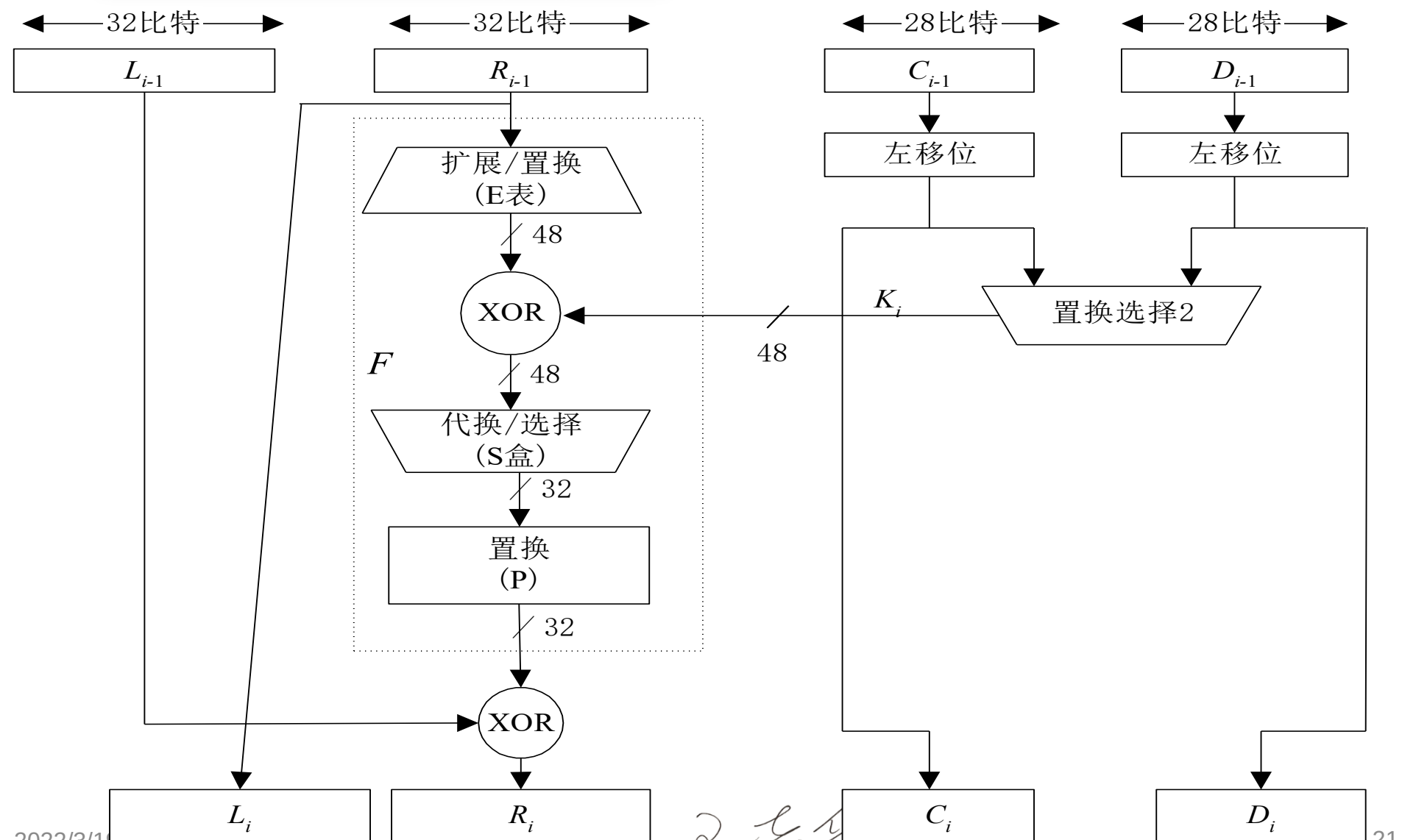


$$M_{20} \rightarrow M'_{14}$$

$$M'_{14} \rightarrow M''_{20}$$



5、DES轮结构



2022/3/19



DES轮结构左半部分:

DES的左半部分

- 将64比特的轮输入分为各为32比特的左、右两半，分别记为L和R。
- 和Feistel网络一样，每轮变换由以下公式表示：

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

- 其中轮密钥为48比特，轮函数 $F(R, K)$ 的计算如图3-7所示。

王华



DES轮结构一轮函数F:

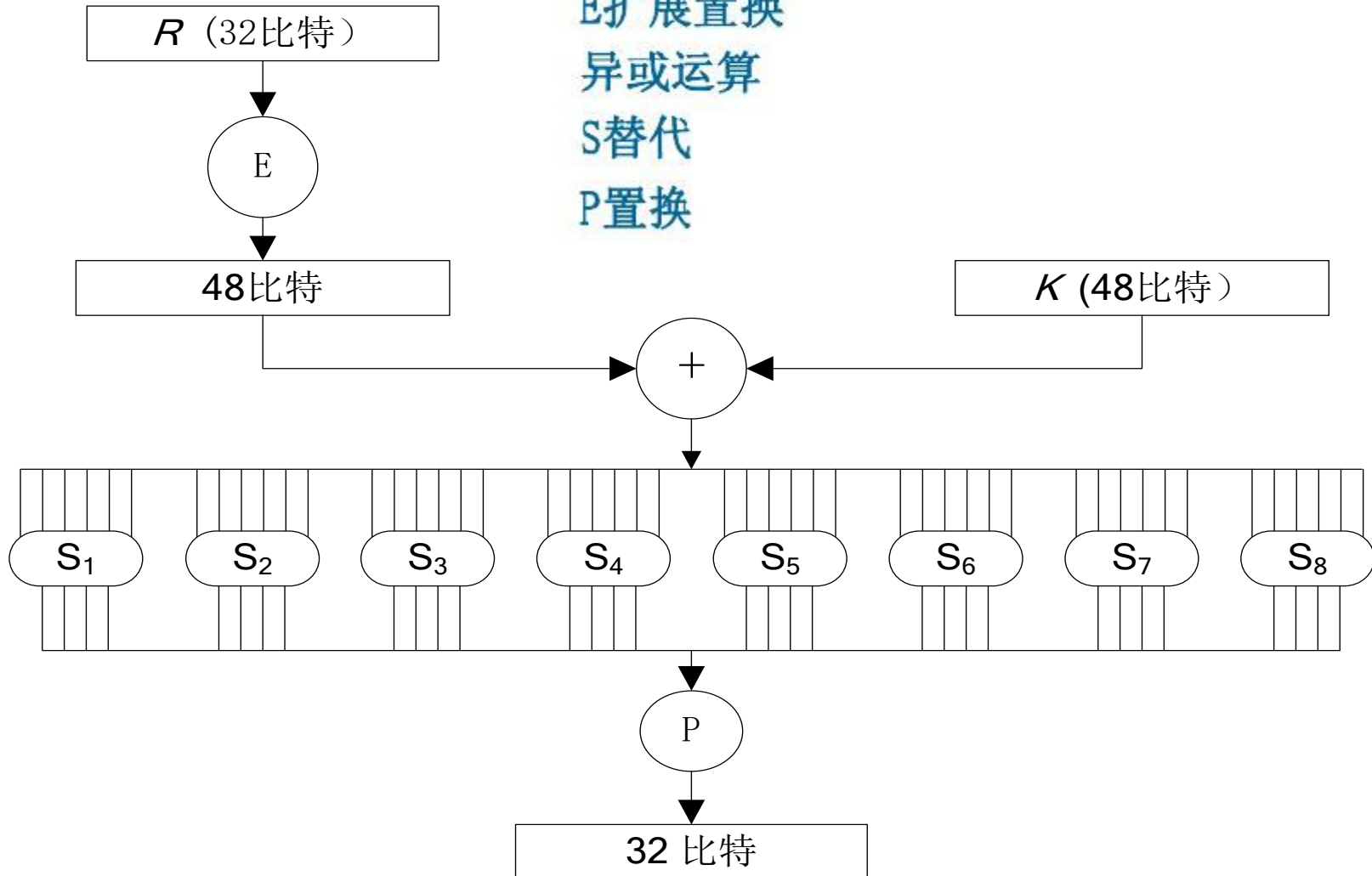
f函数由四个处理过程组成:

E扩展置换

异或运算

S替代

P置换



2022/3/19



DES轮结构—轮函数F:

● 轮函数:

- ① 轮输入R为32比特。
- ② R首先被扩展成48比特，扩展过程由表3-2 (c) 定义，其中将R的16个比特各重复一次。
- ③ 扩展后的48比特再与子密钥 K_i 异或。
- ④ 然后再通过一个S盒，产生32比特的输出。
- ⑤ 该输出再经过一个由表3-2 (d) 定义的置换，产生的结果即为函数 $F(R, K)$ 的输出。

王华



轮函数的扩展表E和置换表P:

(c) 选择扩展运算E

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

(d) 置换运算P

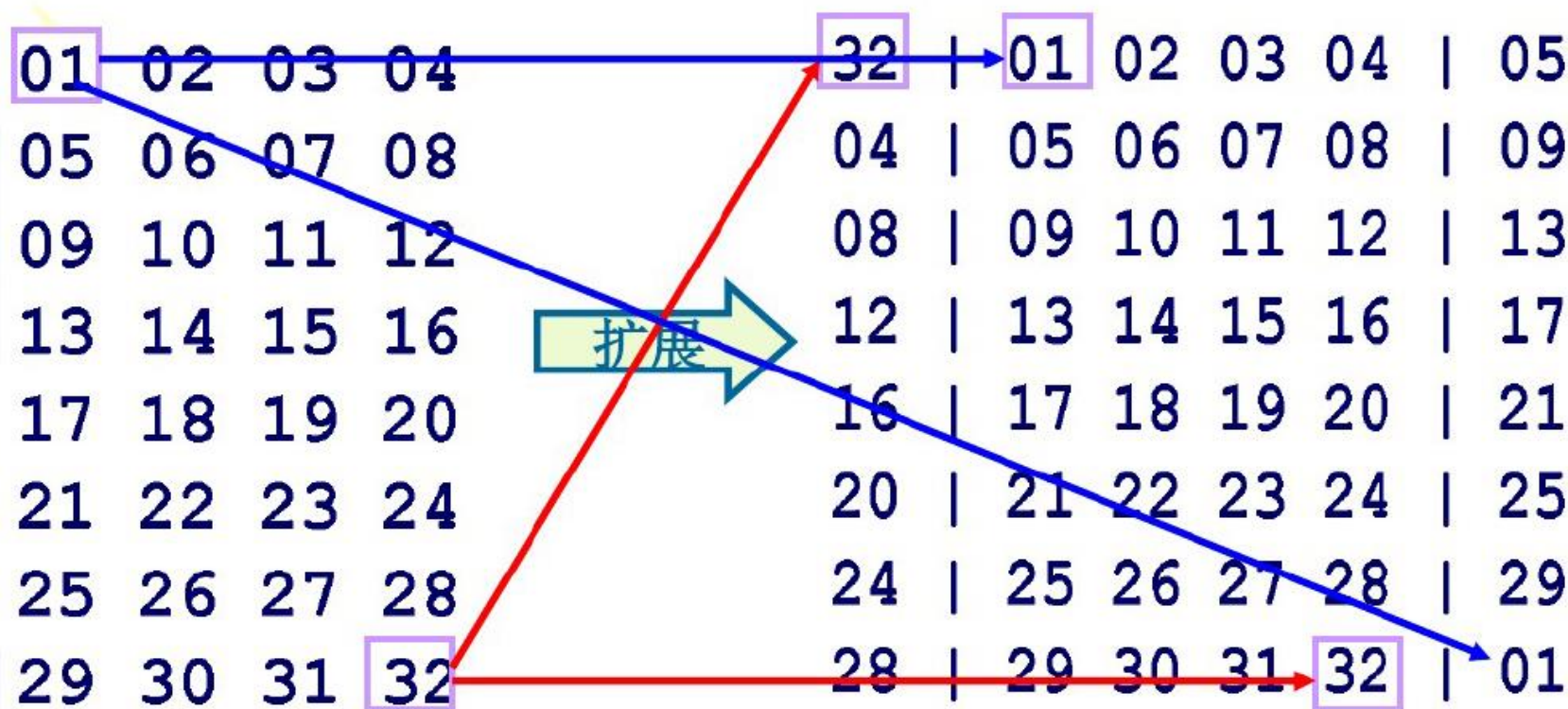
16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

王华



轮函数的扩展运算:

扩展变换的作用是将输入的32比特数据扩展为48比特数据



王立华



轮函数的扩展运算:

将R的16个比特各重复一次

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

扩展方式:

(1) 将输入的32比特每4比特为一组分为8块;

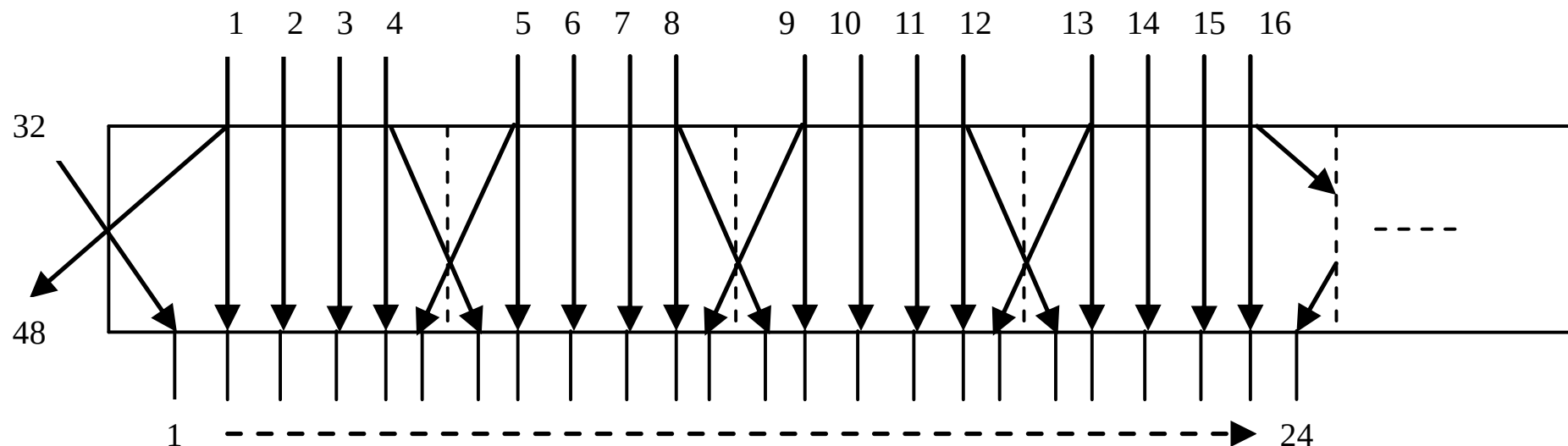
(2) 分别将第 $m-1$ 块的最右比特和第 $m+1$ 块的最左比特添到第 m 块的左边和右边, 形成输出的第 m 个6比特块.

主要原因:硬件实现简单

王卫华

轮函数的扩展运算:

将R的16个比特各重复一次





DES轮结构一轮函数F:

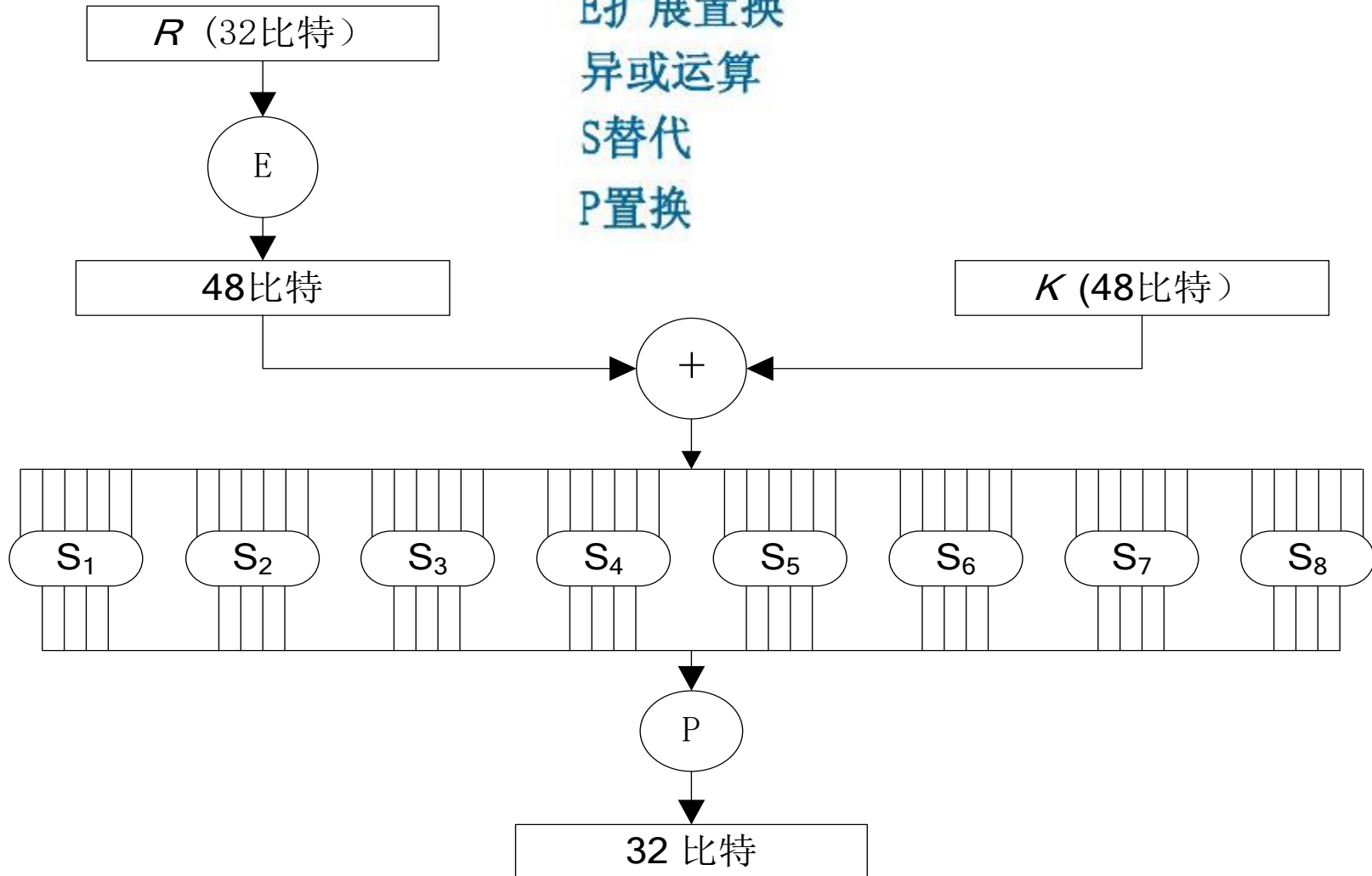
f函数由四个处理过程组成:

E扩展置换

异或运算

S替代

P置换



2022/3/19



DES轮函数—S盒:

● 轮函数中的S盒:

- 轮函数 F 中的代换由**8个S盒**组成。
- 轮函数 F 将48比特压缩到32比特。
- 每个S盒的**输入长为6比特、输出长为4比特**（压缩），其变换关系由表3-3定义。
- **每个S盒给出了4个代换**（由一个表的4行给出），行为2比特、列为4比特（共6比特）。

S盒作为该密码体制的唯一非线性组件对安全性至关重要

S-盒的设计准则还没有完全公开，一些密码学家怀疑美国NSA(the National Security Agency)设计S-盒时隐藏了陷门，即万能钥匙

王华



轮函数中的S盒（仅仅给出S1和S2）：

行\列		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S1	0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S2	0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	1	3	13	4	7	15	2	8	15	12	0	1	10	6	9	11	5
	2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
	3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9

2022/3/19

输出列 行		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S_1	0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2	0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
	2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
	3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3	0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
	1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
	2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
	3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S_4	0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
	1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
	2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
	3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S_5	0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
	1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
	2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
	3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6	0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
	1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
	2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
	3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7	0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
	1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
	2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
	3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8	0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
	1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
	2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
	3	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11



DES轮函数—S盒:

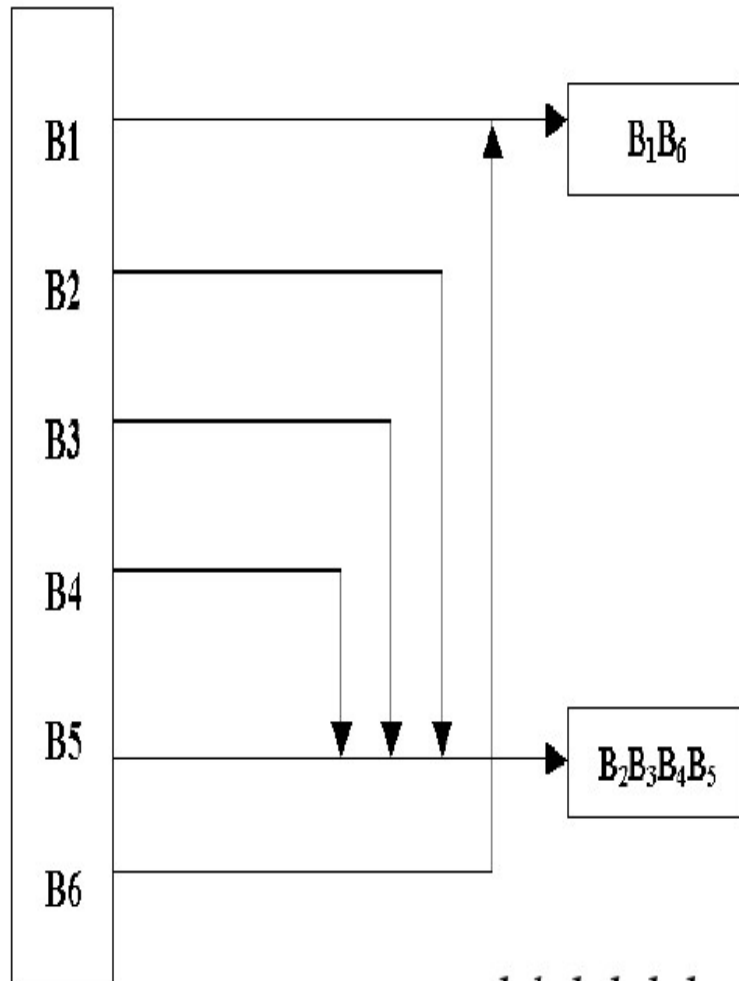
● 轮函数中的S盒:

- 对每个盒 S_i ，其6比特输入中，第1个和第6个比特形成一个2位二进制数，用来选择 S_i 的四个代换中的一个。
- 6比特输入中，中间4位用来选择列。
- 行和列选定后，得到其交叉位置的十进制数，表示为4位二进制数即得这一S盒的输出。
- 例如， S_1 的输入为011001，行选为01（即第1行），列选为1100（即第12列），行列交叉位置的数为9，其4位二进制表示为1001，所以 S_1 的输出为1001。

王立华



DES轮函数—S盒:



行

行\列	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13

列

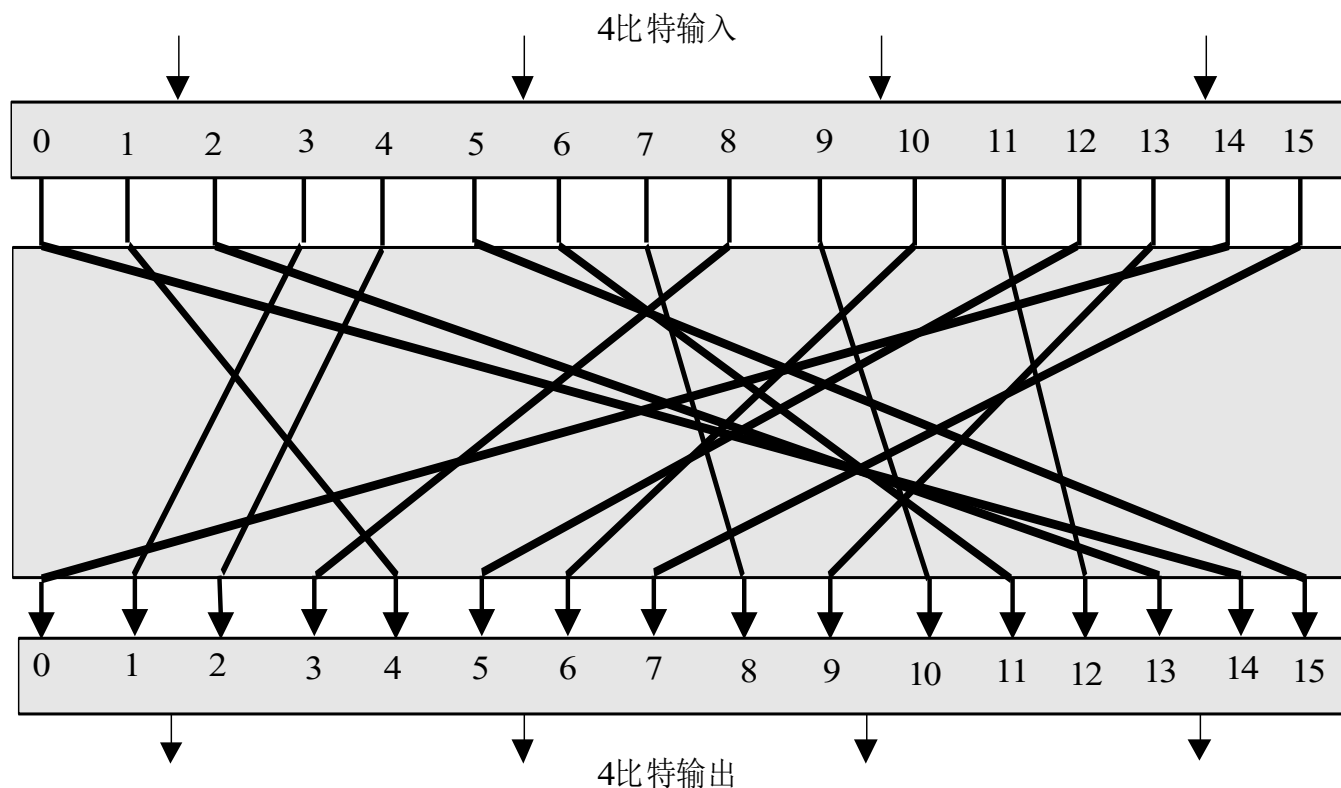
$$\begin{array}{l} b_1 b_2 b_3 b_4 b_5 b_6 \\ \boxed{110011} \end{array} \Rightarrow \begin{array}{l} \text{行: } b_1 b_6 = 11_2 = 3 \\ \text{列: } b_2 b_3 b_4 b_5 = 1001_2 = 9 \end{array} \Rightarrow \begin{array}{l} S_6\text{-盒子 } 3\text{行}9\text{列} \\ \text{值: } 14 = \boxed{1110}_{14} \end{array}$$



DES轮函数—S盒:

● 轮函数中的S盒:

- S盒的4行中的每一行定义了一个可逆代换，下图表示S1第0行所定义的代换。



王立华



DES轮函数—S盒:

● 轮函数中的S盒的构造要求:

- S盒是算法的**唯一非线性部件**，因此它的密码强度决定了整个算法的安全强度;
- S盒提供了密码算法所必须的**混淆**作用;
- S盒要求**非线性度、差分均匀性、严格雪崩准则、可逆性、没有陷门**等。

王华



DES轮结构一轮函数F:

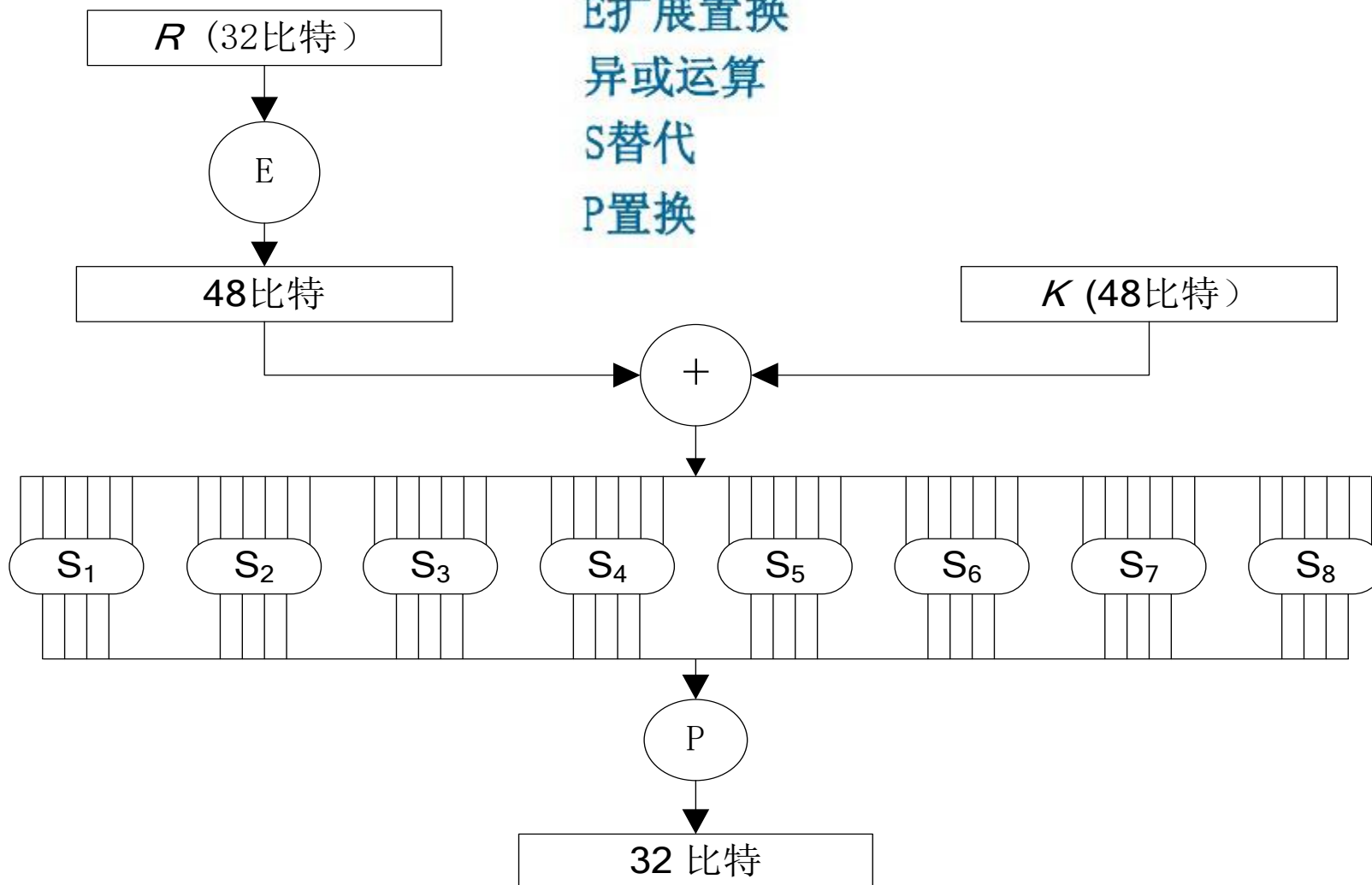
f函数由四个处理过程组成:

E扩展置换

异或运算

S替代

P置换



2022/3/19



DES轮函数—P盒:

将S-盒变换后的32比特数据再进行P盒置换，置换后得到的32比特即为 f 函数的输出。

基本特点:

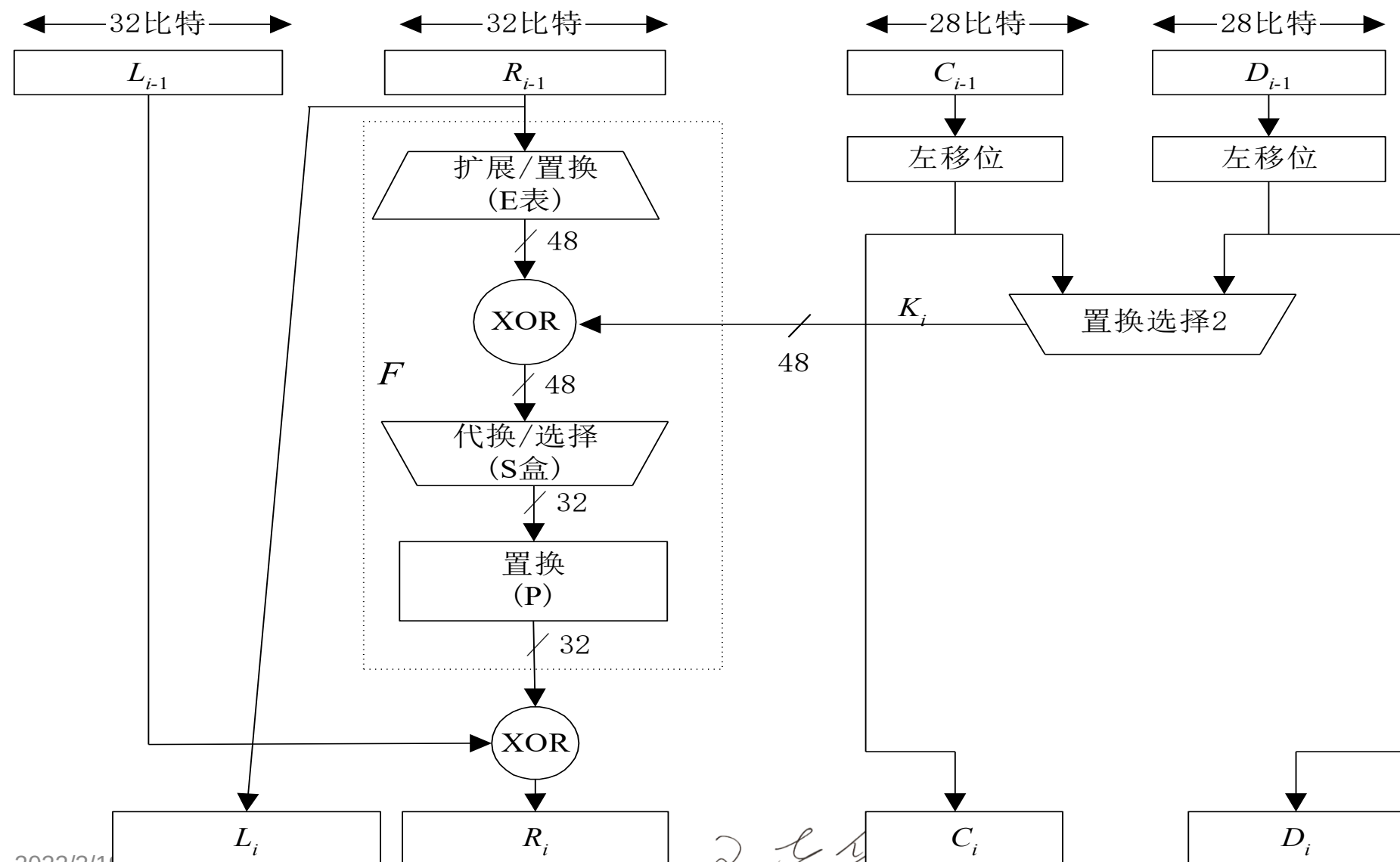
(1) P盒的各输出块的4个比特都来自不同的输入块;

(2) P盒的各输入块的4个比特都分配到不同的输出块之中;

(3) P盒的第 t 输出块的4个比特都不来自第 t 输入块。

P			
16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

含义:P盒输出的第1个元是输入的第16个元。



2022/3/15

例：利用DES算法和全0密钥对输入

10000001 19600000

进行1圈加密的结果。

(1) 输入的右半部分是

19600000 = 0001 1001 0110 0000 0000 0000 0000 0000

(2) 经E盒扩展后变为

000011 110010 101100 000000 000000 000000 000000

(3) 与全0子密钥模2加后变为

000011 110010 101100 000000 000000 000000 000000

(4) 查S盒后的32比特输出为 *f 8 3 7 2 c 4 d*

1111 1000 0011 0111 0010 1100 0100 1101

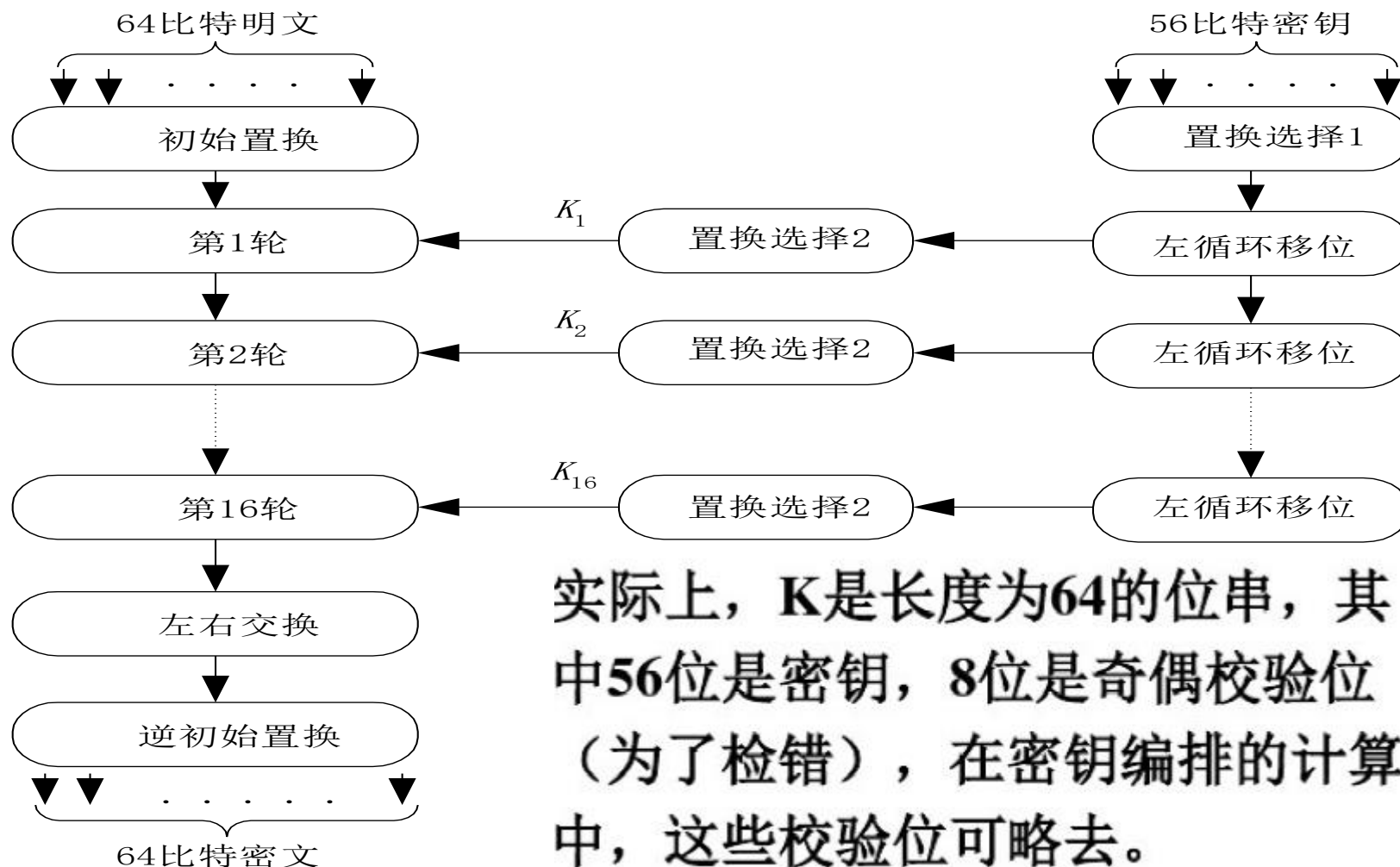
(5) 经P盒后得F函数的32比特输出: *9cd89aae* =

1001 1100 1101 1000 1001 1010 1010 1110

(6) 将F函数的输入放到左边,将F函数的输出与圈函数的左半输入模2加后放到右边,的圈函数的64比特输出为 **19600000 8cd89aaf**



6、DES密钥产生

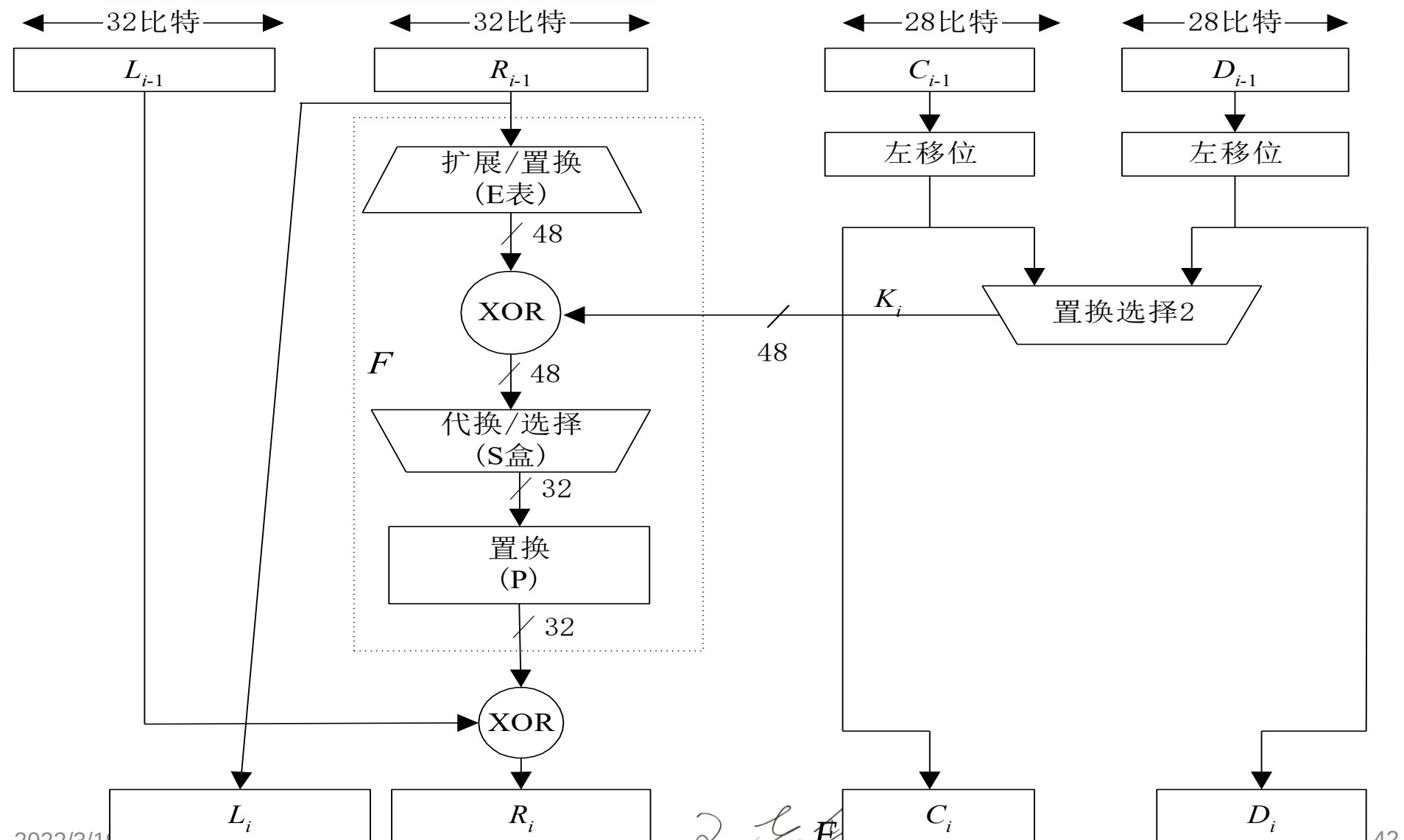


实际上，K是长度为64的位串，其中56位是密钥，8位是奇偶校验位（为了检错），在密钥编排的计算中，这些校验位可略去。

$$E(R_{i-1}, K_i)$$



6、DES密钥产生





DES轮密钥编排:



密钥的产生:

- 算法输入**64比特密钥**。
- 首先经过一个**置换运算**—置换选择1，由表3-4 (a) 给出。
- 然后将置换后的56比特分为**各为28比特的左、右两半**，分别记为 C_0 和 D_0 。
- 在第 i 轮分别对 C_{i-1} 和 D_{i-1} 进行**左循环移位**，**所移位数**由表3-4 (c) 给出。
- 移位后的结果做为求**下一轮子密钥的输入**，同时也作为**置换选择2的输入**。
- 通过表3-4 (b) 的**置换选择2**产生的**48比特的 K_i** ，即为本轮的子密钥，作为函数 $F(R_{i-1}, K_i)$ 的输入。

PC-1						
57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

64 位密钥

置换选择 1

C_0 (28 位)

D_0 (28 位)

循环左移

循环左移

C_1 (28 位)

D_1 (28 位)

循环左移

循环左移

C_i (28 位)

D_i (28 位)

(56 位) 置换选择 2 k_1 (48 位)

(56 位) 置换选择 2 k_i (48 位)

密钥表的计算逻辑

循环左移:

1	1	9	1
2	1	10	2
3	2	11	2
4	2	12	2
5	2	13	2
6	2	14	2
7	2	15	2
8	2	16	1

PC-2

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32



DES轮密钥编排:

(a) 置换选择1

PC-1						
57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

置换选择1:

从64比特密钥中取出56个有效比特。

(b) 置换选择2

PC-2					
14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

置换选择2:

从56个密钥比特中取出48个作为子密钥

2E-1



DES轮密钥编排:

表3-4 DES密钥编排中使用的表

(c) 左循环移位位数

轮数	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
位数	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

意想不到的效果:

子密钥寄存器的内容经16次迭代后又回到原来的设置。

王卫华



7、DES解密

- 和Feistel密码一样，DES的解密和加密使用同一算法，但子密钥使用的顺序相反。

DES 的解密过程可描述为：

$$\begin{cases} R_{i-1} = L_i \\ L_{i-1} = R_i \oplus f(L_i, K_i), i = 16, 15, \dots, 1 \end{cases}$$

解密结构与加密结构完全相同，只不过是所使用的子密钥的顺序正好相反！

加密子密钥： $k_1, k_2, \dots, k_{15}, k_{16}$

解密子密钥： $k_{16}, k_{15}, \dots, k_2, k_1$



DES解密:

$$\mathbf{L}_i = \mathbf{R}_{i-1}, \mathbf{R}_i = \mathbf{L}_i \oplus \mathbf{f}(\mathbf{R}_{i-1}, \mathbf{K}_i)$$

最后一轮迭代后，左右两个**32**比特串并不交换

解密：输入密文经过置换得到比特串 $\mathbf{R}_{16}\mathbf{L}_{16}$ ，
进入迭代第一步输入 $\mathbf{R}_{16}\mathbf{L}_{16}$ ，输出 $\mathbf{L}_{16}, \mathbf{R}_{16} \oplus$
 $\mathbf{f}(\mathbf{L}_{16}, \mathbf{K}_{16})$ ，从加密过程可知 $\mathbf{L}_{16} = \mathbf{R}_{15}, \mathbf{R}_{16} =$
 $\mathbf{L}_{15} \oplus \mathbf{f}(\mathbf{R}_{15}, \mathbf{K}_{16})$

因此有

$$\begin{aligned} \mathbf{R}_{16} \oplus \mathbf{f}(\mathbf{L}_{16}, \mathbf{K}_{16}) &= \mathbf{L}_{15} \oplus \mathbf{f}(\mathbf{R}_{15}, \mathbf{K}_{16}) \oplus \mathbf{f}(\mathbf{L}_{16}, \mathbf{K}_{16}) \\ &= \mathbf{L}_{15} \end{aligned}$$



8、DES总结

(1) 密码模型

● Feistel模型

(3) 基本参数

- 分组长度：64比特
- 密钥长度：64比特
- 有效密钥长度：56比特
- 迭代圈数：16圈
- 每圈子密钥长度：48比特

实现性能：

软件实现慢、硬件实现快；可全部用布尔电路实现

(2) F函数模型

● S-----P网络

(4) 编码环节

- 6进4出的S盒变换
- 逐位模2加变换 $\oplus$
- 比特移位变换P盒和IP
- 比特扩展变换E盒
- 比特抽取变换PC₁、PC₂

2022



9、DES安全性分析

完全依赖于所用的密钥，即算法是公开的

DES的设计特色：

在**DES**算法中函数是最基本的关键环节，其中

- (1) 用**S**盒实现小块的非线性变换，达到混乱目的
- (2) 用置换**p**实现大块的非线性变换，达到扩散目的
- (3) 置换**p**的设计是每层**s**盒的**4bit**输出进入到下一层的**4**个不同**S**盒

2022/3/19



DES安全性

- DES的安全性设计：
 - 密文与明文、密文与密钥的相关性
 - 代数结构分析
 - 密码分析
- DES的安全性问题：
 - 互补性（取反特性）
 - F函数（S-Box）设计原理未知
 - 密钥长度
 - 弱密钥与半弱密钥。

王华



DES安全性设计:

密文与明文、密文与密钥的相关性。

Meyer[1978]详细研究了DES的输入明文与密文及密钥与密文之间的相关性。表明每个密文比特都是所有明文比特和所有密钥比特的复合函数，并且指出达到这一要求所需的迭代次数至少为**5**。**Konheim[1981]**用 χ^2 检验证明，迭代**8**次后输出和输入就可认为是不相关的了。

2022/3/19



DES安全性设计:

●代数结构分析:

- 1992年证明了DES不是一个群，这说明DES具有较好的密码学性质
- IBM密码研究小组的Coppersmith宣称，他们早就知道DES不是一个群

●密码分析

1990年，以色列密码学家**Eli Biham**和**Adi Shamir**提出了差分密码分析法，可对**DES**进行选择明文攻击

线性密码分析比差分密码分析更有效

王华



DES密码分析：线性密码分析

- 该方法是Mitsuru Matsui于1993年公开的另一种对分组密码进行分析攻击的方法，这种方法试图通过大量的“明---密文对”找出分组密码算法中与密钥有关的线性方程，然后试着得到大量的这类关系从而确定密钥。
- 其基本思想是以最佳的线性函数逼近DES的非线性变换S盒，这是一种已知明文攻击方法，可以在有 2^{43} 个已知明文的情况下破译DES。虽然获得已知明文比选择明文更容易，但线性分析作为一种攻击手段在实际上仍然不可行。

2022/3/19



DES密码分析：差分密码分析

- 差分分析是一种攻击迭代密码体制的选择明文攻击方法，与一般统计分析法的不同之处是，它不是直接分析密文或密钥与明文的统计相关性，而是分析一对给定明文的异或（称为差分）与对应密文对的差异之间的统计相关性。
- 差分分析的基本思想是在要攻击的迭代密码系统中找出某些高概率的明文差分 and 密文差分对来推算密钥。利用此法攻击DES，需要用**2⁴⁷个选择明文和2⁴⁷次加密运算**，比穷举搜索的工作量大减少。然而找到2⁴⁷个选择明文的要求使这种攻击只有理论上的意义。

王华



DES安全性问题:

互补性。DES算法具有下述性质。若明文组 x 逐位取补，密钥 k 逐位取补，即 $y = \text{DES}_k(x)$ ，则有 $\bar{y} = \text{DES}_{\bar{k}}(\bar{x})$

这种互补性会使DES在选择明文破译下所需的工作量减半。

2022/3/19



DES安全性问题:

● S盒的设计:

- 安全的主要依据是**依赖于S盒（非线性）**，其他部件均为线性部件
- S盒的设计原理未知，被怀疑隐藏了“陷门”
- 目前未知，并未发现有所谓的“陷门”
- S盒在最初就被设计成可以防止某些类型的攻击

王卫华



DES安全性问题:

❖ 密钥空间过小:

- 对DES安全性批评意见中，较为一致的看法是**DES的密钥短了些**。
- IBM最初向NBS提交的建议方案采用112 比特密钥，但公布的DES标准采用64比特密钥。因此，有人认为NSA故意限制DES的密钥长度。
- 采用密钥穷举搜索已经对DES构成了威胁。

王华



DES安全性问题:

关于**DES**算法的另一个最有争议的问题就是担心实际**56**比特的密钥长度不足以抵御穷举式攻击，因为密钥量只有 $2^{56} \approx 10^{17}$ 个

早在**1977**年，**Diffie**和**Hellman**已建议制造一个每秒能测试**100**万个密钥的**VLSI**芯片。每秒测试**100**万个密钥的机器大约需要一天就可以搜索整个密钥空间。他们估计制造这样的机器大约需要**2000**万美元

在**CRYPTO'93**上，**Session**和**Wiener**给出了一个非常详细的密钥搜索机器的设计方案，这个机器基于并行运算的密钥搜索芯片，所以**16**次加密能同时完成。花费**10**万美元，平均用**1.5**天左右就可找到**DES**密钥

美国克罗拉多洲的程序员**Verser**从**1997**年**2**月**18**日起，用了**96**天时间，在**Internet**上数万名志愿者的协同工作下，成功地找到了**DES**的密钥，赢得了悬赏的**1**万美元



DES安全性问题:

1998年7月电子前沿基金会（**EFF**）使用一台**25万**美圆的电脑在**56**小时内破译了**56**比特密钥的**DES**

1999年1月RSA数据安全会议期间，电子前沿基金会用**22**小时**15**分钟就宣告破解了一个**DES**的密钥

2022/3/19



DES安全性问题:

● DES存在弱密钥:

- 所谓的弱密钥是指在所有可能的密钥中, 有几个特别的密钥会降低DES的安全性, 所以使用者一定要避免使用这几个弱密钥。
- DES存在着一些**弱密钥**, **这些密钥使得每一轮的子密钥相同**。

$$\text{DES}_K(\text{DES}_K(m)) = m$$

$$\text{DES}_K^{-1}(\text{DES}_K^{-1}(m)) = m$$

$k_1 = k_2 = \dots = k_{16}$, 满足

$$\text{DES}_k(\text{DES}_k(m)) = m \text{ 或 } \text{DES}_k(m) = \text{DES}_k^{-1}(m)$$

2024



DES安全性问题:

若 k 为弱密钥, 则有

$$\text{DES}_k(\text{DES}_k(x))=x$$

$$\text{DES}_k^{-1}(\text{DES}_k^{-1}(x))=x$$

即以 k 对 x 加密两次或解密两次都可恢复出明文。其加密运算和解密运算没有区别。

弱密钥下使DES在选择明文攻击下的搜索量减半。

如果随机地选择密钥, 弱密钥所占比例极小, 而且稍加注意就不难避开。因此, 弱密钥的存在不会危及DES的安全性。

● 目前发现的弱密钥只有四个: 全0、全1, 半0, 半1。

王华



DES安全性问题:

● DES存在半弱密钥:

- DES还存在着一些**半弱密钥**，这些密钥产生的子密钥有很多是相同的。

若存在初始密钥 K 和 $K' (K' \neq K)$ ，使 $\text{DES}_K(\text{DES}_{K'}(m)) = m$ ，则称 K 是半弱密钥，且称 K' 与 K 是对合的。

- 半弱密钥是成对出现的，共6对十二个。

王华



DES安全性问题:

半弱密钥对(每行一对)	
01FE01FE01FE01FE	FE01FE01FE01FE01
1FE01FE00EF10EF1	E01FE01FF10EF10E
01E001E001F101F1	E001E001F101F101
1FFE1FFE0EFE0EFE	011F011F010E010E
FE1FFE1FFE0EFE0E	1F011F010E010E01
E0FEE0FEF1FEF1FE	FEE0FEE0FEF1FEF1

2022/3/19



DES安全性问题:

- DES是现代对称分组密码体制的代表
- 目前56位密钥的单DES已不安全，美国在80年代末即可在一周内破译DES
- 93年美国可在3.5小时内破译DES
- 现在美国可在5分钟内破译DES
- 美国政府宣布98年以后不再使用DES
- 为了提高安全性，现在通常都使用112位或168位密钥的3-DES

王华



2.3.2 二重DES

1、二重DES概述

- 为了提高DES的安全性，并利用实现DES的现有软硬件，可将DES算法在**多密钥**下多重使用。
- 二重DES是**多重使用DES**时最简单的形式。

2022/3/19

2022/3/19



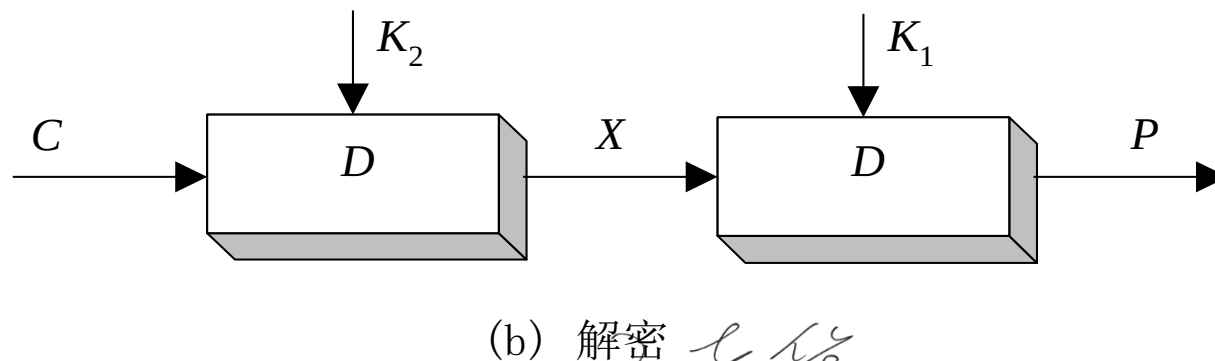
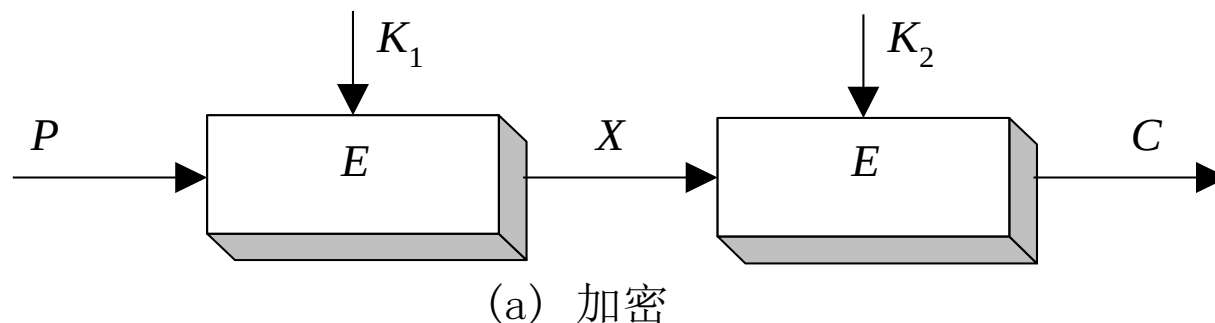
2、二重DES基本思想

- 加密时，明文为 P ，**两个加密密钥为 K_1 和 K_2** ，密文为：

$$C = E_{K_2}[E_{K_1}[P]]$$

- 解密时，以相反顺序使用两个密钥：

$$P = D_{K_1}[D_{K_2}[C]]$$



赵志华



3、二重DES安全性分析

- 二重DES所用**密钥长度为112比特**，强度极大地增加。
- 但是，如果对任意两个密钥 K_1 和 K_2 ，能够找出另一密钥 K_3 ，使得下式成立，则二重DES以及多重DES都没有意义，因为它们与56比特密钥的单重DES等价。

$$E_{K_2}[E_{K_1}[P]] = E_{K_3}[P]$$

2022/3/19



二重DES加密—安全性分析:

● 但幸运的是，**上式对DES并不成立**。

- 将DES加密过程64比特分组到64比特分组的映射看作一个置换，如果考虑 2^{64} 个所有可能的输入分组，则密钥给定后，DES的加密将把每个输入分组映射到一个惟一的输出分组。
- 否则，如果有两个输入分组被映射到同一分组，则解密过程就无法实施。
- 对 2^{64} 个输入分组，总映射个数为 $(2^{64})! > (10^{10^{20}})$ 。
- 另一方面，对每个不同的密钥，DES都定义了一个映射，总映射数为 $2^{56} < 10^{17}$ 。

● 因此，可假定用两个不同的密钥两次使用DES，可得一个新映射，而且这一新映射不出现在单重DES定义的映射中。

● 所以，**使用二重DES产生的映射不好等价于单重DES加密**。

2.2.4



3、二重DES加密—中途相遇攻击

● 但对二重DES有以下一种称为**中途相遇攻击**的攻击方案，这种攻击不依赖于DES的任何特性，因而可用于**攻击任何分组密码**。

● 其基本思想如下：

➤ 如果有： $C = E_{K_2}[E_{K_1}[P]]$

➤ 那么有： $X = E_{K_1}[P] = D_{K_2}[C]$

中途相遇攻击法(Meet-in-the-Middle Attack)

由Diffie和Hellman[1977]最早提出，可以降低搜索量其基本想法如下。若有明文密文对 (x_i, y_i) 满足

$$y_i = E_{k_2}[E_{k_1}[x_i]]$$

$$z = E_{k_1}[x_i] = D_{k_2}[y_i]$$

则可得



二重DES加密—中途相遇攻击过程:

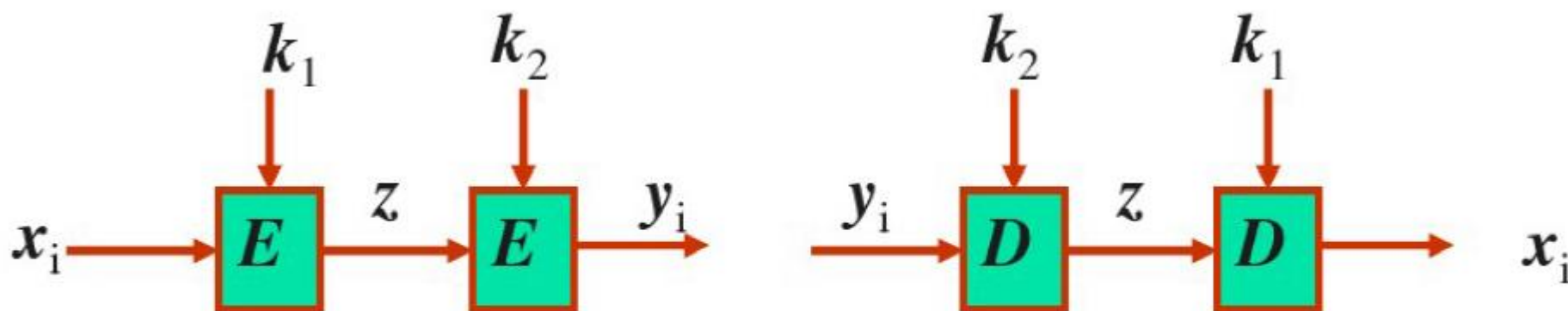


图4-14-1 中途相遇攻击示意图

2022/3/19



二重DES加密—中途相遇攻击过程:

- 如果已知一个明文密文对 (P, C) , 可实施可如下攻击:
 - 首先, 用 2^{56} 个所有可能的 K_1 对 P 加密, 将加密结果存入一表并对表按 X 排序;
 - 然后用 2^{56} 个所有可能的 K_2 对 C 解密, 在上述表中**查找**与 C 解密结果相匹配的项;
 - ✓ 如果找到, 则记下相应的 K_1 和 K_2 。
 - 最后再用一新的明文密文对 (P', C') 检验上面找到的 K_1 和 K_2 , 用 K_1 和 K_2 对 (P') 两次加密, 若结果等于 C' , 就可确定 K_1 和 K_2 是所要找的密钥。

王华



二重DES加密—中途相遇攻击分析:

- 对已知的明文 P , 二重DES能产生 2^{64} 个可能的密文。
- 可能的密钥个数为 2^{112} , 所以平均来说, 对一个已知的明文, 有 $2^{112}/2^{64} = 2^{48}$ 个密钥可产生已知的密文。
- 再经过另外一对明文密文的检验, 误报率将下降到 $2^{48-64} = 2^{-16}$ 。
- 所以在实施中途相遇攻击时, 如果已知两个明文密文对, 则找到正确密钥的概率为 $1 - 2^{-16}$ 。

这一攻击法所需的存储量为 $2^{56} \times 8$ Byte, 最大试验的加密次数 $2 \times 2^{56} = 2^{57}$ 。这说明破译双重DES的难度为 2^{57} 量级。

王华



2.3.3 三重DES

1、三重DES概述

- 抵抗中途相遇攻击的一种方法是**使用3个不同的密钥做3次加密**，从而可使已知明文攻击的代价增加到 2^{112} 。
- 然而，这样又会使密钥长度增加到 $56 \times 3 = 168$ 比特，因而过于笨重。
- 所以，考虑使用**三重DES加密**。

王华



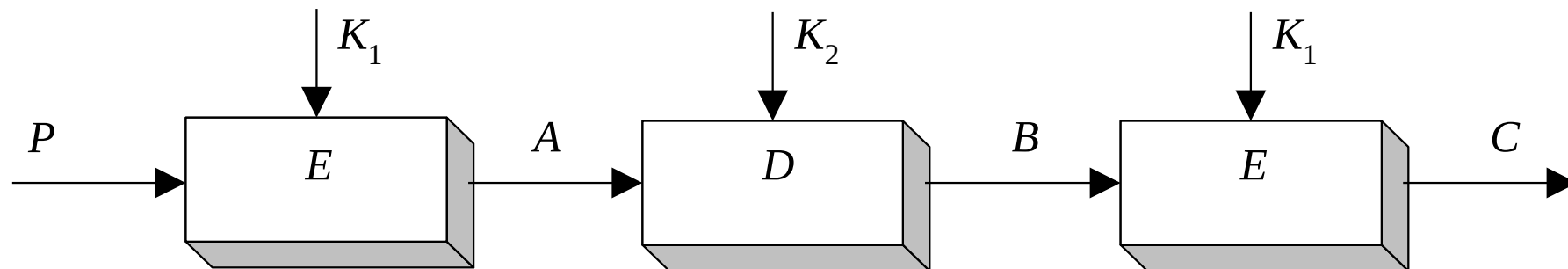
2、2个密钥的三重DES：方法1

- 一种实用的方法是仅**使用2个密钥做三次加密**，实现方式为**加密——解密——加密**，简记为EDE(Encrypt- Decrypt- Encrypt)。
- 加密时： $C = E_{K_1}[D_{K_2}[E_{K_1}[P]]]$
 - 第2步解密的目的仅在于使得用户可对一重DES加密的数据解密。
- 此方案已在密钥管理标准ANS X.917和ISO 8732中被采用。

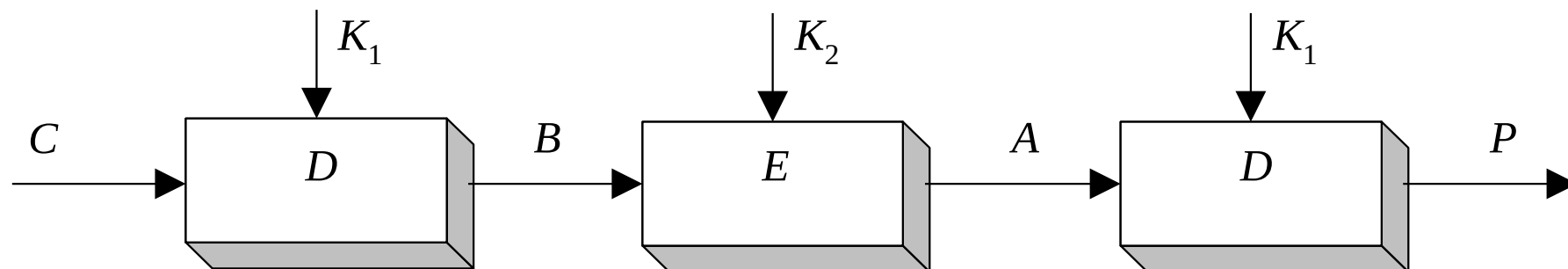
2022/3/19



两个密钥112位



(a) 加密



(a) 解密

两个密钥的三重DES

王华



3、3个密钥的三重DES：方法2

- 另一种方法是**使用3个密钥做三次加密**，三个密钥的三重DES密钥长度为168比特。
- 加密时：

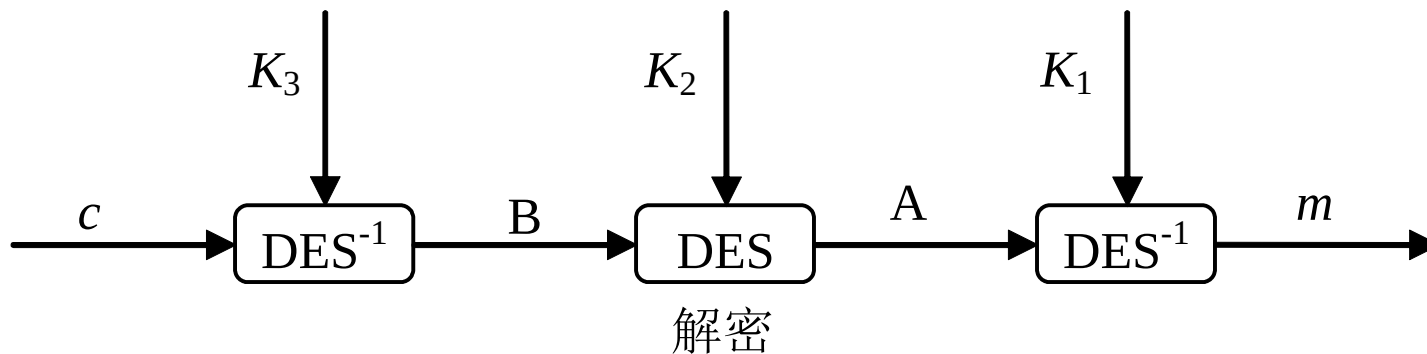
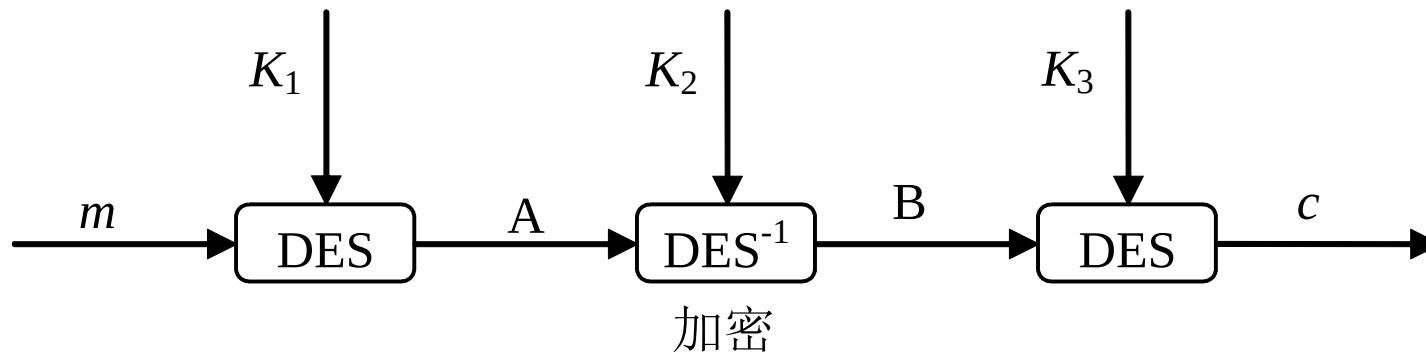
$$C = E_{K_3} [D_{K_2} [E_{K_1} [P]]]$$

- 令 $K_3 = K_2$ 或 $K_1 = K_2$ ，则变为一重DES。
- 三个密钥的三重DES已在因特网的许多应用（如PGP和S/ MIME）中被采用。

2022/3/19



三个密钥168位



三个密钥的三重DES

这是EDE模式，类似地有EEE模式。

王华

2.3.4 DES密码总结

- DES是第一个公开的国际标准分组算法。
- DES目前已经不再单独使用，但3-DES仍出现在许多安全协议的备选算法中。
- 尽管DES已经被更新的算法所替代，但它在密码学的发展史上有其独特的标志性的地位。
- DES的设计思想仍在后来的算法中屡屡使用。

2022/3/19